

Jobsheet 5 – Brute Force dan Divide Conquer0

Nama : Muhammad Fauzi Fadillah

NIM : 254107020085

Kelas : TI_1G

5.2 Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer

5.2.1 Langkah-langkah Percobaan

Code program Faktorial.java

```
package minggu5.BruteForceDivideConquer;
```

```
public class Faktorial {
```

```
    int faktorialBF(int n){
```

```
        int fakto = 1;
```

```
        for (int i = 1; i <= n; i++) {
```

```
            fakto = fakto * i;
```

```
        }
```

```
        return fakto;
```

```
    }
```

```
    int faktorialDC(int n){
```

```
        if(n == 1){
```

```
            return 1;
```

```
        }else{
```

```
            int fakto = n * faktorialDC(n-1);
```

```
            return fakto;
```

```
    }  
    }  
}
```

Code program MainFaktorial.java

```
package minggu5.BruteForceDivideConquer;
```

```
import java.util.Scanner;
```

```
public class MainFaktorial {
```

```
    public static void main(String[] args) {
```

```
        Scanner input = new Scanner(System.in);
```

```
        System.out.print("Masukkan nilai: ");
```

```
        int nilai = input.nextInt();
```

```
        Faktorial fk = new Faktorial();
```

```
        System.out.println("Nilai faktorial " + nilai +
```

```
            " menggunakan BF: " + fk.faktorialBF(nilai)
```

```
        );
```

```
        System.out.println("Nilai faktorial " + nilai +
```

```
            " menggunakan DC: " + fk.faktorialDC(nilai)
```

```
        );
```

```
        input.close();
```

```
    }
```

```
}
```

5.2.2 Verifikasi Hasil Percobaan

```
Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
```

5.2.3 Pertanyaan

Jawaban

1. Kode if digunakan untuk base case atau untuk membatasi proses rekursi, sedangkan else merupakan recursion call dimana proses pemanggilan fungsi itu sendiri yang digunakan untuk proses pengulangan rekursi

2. Bisa dengan menggunakan while

Code program fungsi faktorialBF

```
int faktorialBF(int n){
    int fakto = 1;
    int i = 1;
    //for (int i = 1; i <= n; i++) {
    //    fakto = fakto * i;
    //}

    while(i <= n){
        fakto = fakto * i;
        i++;
    }

    return fakto;
}
```

output:

```
Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
```

3. Perbedaan fakto *= i dan int fakto = n * faktorialDC(n-1);
 - a. fakto *= i : proses ini digunakan untuk algoritma Brute Force yang dimana menggunakan for loop, jadi pengulangannya disimpan di variabel fakto diluar looping. Untuk proses faktorial diurutkan dari iterasi 1 yang merupakan bilangan 1 juga, lalu proses fakto*= i selama i kurang dari atau sama dengan jumlah n
 - b. n * faktorialDC(n-1) : proses ini digunakan untuk algoritma Divide and Conquer. Posisi dari proses tersebut ada berada di kondisi else yang dimana merupakan recursion call, yaitu proses rekursi yang memanggil fungsi itu

sendiri. Maka, proses ini digunakan untuk menghitung faktorial dari n dikali dengan pemanggilan fungsi sendiri tetapi parameter n dikurangi 1

4. faktorialBF() bekerja secara iteratif dengan loop, sedangkan faktorialDC() bekerja secara rekursif dengan memanggil method sendiri sampai mencapai base case. Keduanya menghasilkan nilai yang sama, tetapi mekanisme prosesnya berbeda.

5.3 Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer

5.3.1 Langkah-langkah Percobaan

Code program Pangkat.java

```
package minggu5;
```

```
public class Pangkat {
```

```
    int nilai, pangkat;
```

```
    public Pangkat(int n, int p){
```

```
        nilai = n;
```

```
        pangkat = p;
```

```
    }
```

```
    int pangkatBF(int a, int n){
```

```
        int hasil = 1;
```

```
        for (int i = 0; i < n; i++) {
```

```
            hasil = hasil * a;
```

```
        }
```

```
        return hasil;
```

```
    }
```

```
    int pangkatDC(int a, int n){
```

```

    if(n == 1){
        return a;
    }else{
        if(n % 2 == 1){
            return(pangkatDC(a, n/2) * pangkatDC(a, n/2) * a );
        }else{
            return(pangkatDC(a, n/2) * pangkatDC(a, n/2));
        }
    }
}
}
}
}

```

Code program MainPangkat.java
package minggu5;

import java.util.Scanner;

public class MainPangkat {

public static void main(String[] args) {

Scanner input = new Scanner(System.in);

System.out.print("Masukkan jumlah elemen: ");

int elemen = input.nextInt();

Pangkat[] png = new Pangkat[elemen];

for (int i = 0; i < elemen; i++) {

System.out.print("Masukkan nilai basis elemen ke-" + (i + 1) + ": ");

int basis = input.nextInt();

```
        System.out.print("Masukkan nilai pangkat elemen ke-" + (i + 1) + ": ");
        int pangkat = input.nextInt();

        png[i] = new Pangkat(basis, pangkat);
    }

    System.out.println("HASIL PANGKAT BRUTEFORCE");
    for (Pangkat p : png) {
        System.out.println(p.nilai + "^" + p.pangkat + ": " + p.pangkatBF(p.nilai, p.pangkat));
    }

    System.out.println("HASIL PANGAT DIVIDE CONQUER");
    for (Pangkat p : png) {
        System.out.println(p.nilai + "^" + p.pangkat + ": " + p.pangkatDC(p.nilai, p.pangkat));
    }

    input.close();
}
}
```

5.3.2 Verifikasi Hasil Percobaan

```
Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1: 2
Masukkan nilai pangkat elemen ke-1: 3
Masukkan nilai basis elemen ke-2: 4
Masukkan nilai pangkat elemen ke-2: 5
Masukkan nilai basis elemen ke-3: 6
Masukkan nilai pangkat elemen ke-3: 7
HASIL PANGKAT BRUTEFORCE
2^3: 8
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE CONQUER
2^3: 8
4^5: 1024
6^7: 279936
PS C:\College\semester2\ASD\Praktikum>
```

5.3.3 Pertanyaan

Jawaban

1. pangkatBF() bekerja secara iteratif dengan perulangan. Nilai basis dikalikan berulang sebanyak nilai pangkat. Jadi kalau menghitung 2^5 , method ini akan melakukan perkalian $2 \times 2 \times 2 \times 2 \times 2$.
pangkatDC() bekerja secara rekursif dengan pendekatan divide and conquer. Pangkat dipecah menjadi submasalah yang lebih kecil, yaitu $n/2$, lalu hasilnya digabung lagi.
2. Proses combine di fungsi pangkatDC() ada saat proses return 2 call function dikalikan kembali, dimana dibagi menjadi 2 yaitu ketika parameter n atau pangkat merupakan ganjil (`return(pangkatDC(a, n/2) * pangkatDC(a, n/2) * a );`) dan yang genap (`return(pangkatDC(a, n/2) * pangkatDC(a, n/2));`)
3. Method itu tetap relevan memakai parameter, karena method jadi lebih fleksibel. Method bisa dipakai untuk menghitung pangkat nilai apa pun tanpa harus selalu bergantung pada atribut object.
4. Sama-sama digunakan mencari hasil pangkat tetapi cara kerja berbeda. pangkatBF() menghitung dengan cara mengkalikan baris secara berulang sebanyak nilai pangkat nya menggunakan looping. Sedangkan pangkatDC() menghitung dengan membagi masalah menjadi sub-sub masalah yang akhirnya semua hasilnya digabung dan terlihat hasil akhirnya menggunakan rekursif

5.4 Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer

5.4.1 Langkah-langkah Percobaan

Code program Sum.java

```
package minggu5;
```

```
public class Sum {
```

```
    double keuntungan[];
```

```
    Sum(int el){
```

```
        keuntungan = new double[el];
```

```
    }
```

```
    double totalBF(){
```

```
        double total = 0;
```

```
        for (int i = 0; i < keuntungan.length; i++) {
```

```
            total = total + keuntungan[i];
```

```
        }
```

```
        return total;
```

```
    }
```

```
    double totalDC(double arr[], int l, int r){
```

```
        if(l == r){
```

```
            return arr[l];
```

```
        }
```

```
        int mid = (l + r) / 2;
```

```
        double lsum = totalDC(arr, l, mid);
```



```

        double rsum = totalDC(arr, mid + 1, r);

        return lsum + rsum;
    }
}

```

Code program MainSum.java
package minggu5;

```
import java.util.Scanner;
```

```

public class MainSum {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();

        Sum sm = new Sum(elemen);
        for (int i = 0; i < elemen; i++) {
            System.out.print("Masukkan keuntungan ke-" + (i + 1) + ": ");
            sm.keuntungan[i] = input.nextDouble();
        }

        System.out.println("Total keuntungan menggunakan Bruteforce: " + sm.totalBF());
        System.out.println("Total keuntungan menggunakan Divide and Conquer: " +
sm.totalDC(sm.keuntungan, 0, elemen-1));

        input.close();
    }
}

```

}

5.4.2 Verifikasi Hasil Percobaan

```
Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 10
Masukkan keuntungan ke-2: 20
Masukkan keuntungan ke-3: 30
Masukkan keuntungan ke-4: 40
Masukkan keuntungan ke-5: 50
Total keuntungan menggunakan Bruteforce: 150.0
Total keuntungan menggunakan Divide and Conquer: 100.0
PS C:\College\semester2\ASD\Praktikum>
```

5.4.3 Pertanyaan

Jawaban:

1. Variabel mid digunakan untuk membagi array menjadi 2 bagian, bagian kiri “l” sampai mid dan bagian kanan mid sampai “r”. Jadi, mencari titik tengah lalu titik tengah digunakan untuk membagi data menjadi 2 bagian
2. Menghitung setiap submasalah di bagian kiri dan kanan
3. Fungsi proses tersebut digunakan untuk conquer yaitu menggabungkan hasil dari pemecahan sub masalah
4. Ketika $l == r$. Maksudnya, ketika index yang dibandingkan hanya berjumlah 1 elemen
5. totalDC() berfungsi dengan cara membagi array menjadi 2 bagian kiri dan kanan, lalu menghitung masing-masing bagian tersebut dan hasilnya dijumlahkan di akhir program fungsi.